# LIST COMPREHENSION, FUNCTIONS AS OBJECTS, TESTING, DEBUGGING

(download slides and .py files to follow along)

6.100L Lecture 12

Ana Bell

# LIST COMPREHENSIONS

# LIST COMPREHENSIONS

- Applying a **function to every element of a sequence**, then creating a new list with these values is a common concept
- Example:

New list

Function to apply

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

- Python provides a concise one-liner way to do this, called a **list comprehension**
  - Creates a new list
  - Applies a function to every element of another iterable
  - Optional, only apply to elements that satisfy a test

```
[expression for elem in iterable if test]
```

# LIST COMPREHENSIONS

- Create a **new** list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

New list

Look at every element

Function to apply

```
Lnew = [e**2 for e in L]
```

New list

# LIST COMPREHENSIONS

- Create a **new** list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

```
Lnew = [e**2 for e in L]
```

```
def f(L):
    Lnew = []
    for e in L:
        if e%2==0:
            Lnew.append(e**2)
    return Lnew
```

New list

Function to apply only if test is true

# LIST COMPREHENSIONS

- Create a **new** list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

```
Lnew = [e**2 for e in L]
```

```
def f(L):
    Lnew = []
    for e in L:
        if e%2==0:
            Lnew.append(e**2)
    return Lnew
```

New list

```
Lnew = [e**2 for e in L if e%2==0]
```

# LIST COMPREHENSIONS

- Create a **new** list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

```
Lnew = [e**2 for e in L]
```

```
def f(L):
    Lnew = []
    for e in L:
        if e%2==0:
            Lnew.append(e**2)
    return Lnew
```

Loop over elements

```
Lnew = [e**2 for e in L if e%2==0]
```

# LIST COMPREHENSIONS

- Create a **new** list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

```
Lnew = [e**2 for e in L]
```

```
def f(L):
    Lnew = []
    for e in L:
        if e%2==0:
            Lnew.append(e**2)
    return Lnew
```

Function to apply only if test is true

```
Lnew = [e**2 for e in L if e%2==0]
```

# LIST COMPREHENSIONS

```
[expression for elem in iterable if test]
```

- This is equivalent to invoking this function (where expression is a function that computes that expression)

```
def f(expr, old_list, test = lambda x: True):
    new_list = []
    for e in old_list:
        if test(e):
            new_list.append(expr(e))
    return new_list
```

```
[e**2 for e in range(6)]                → [0, 1, 4, 9, 16, 25]
[e**2 for e in range(8) if e%2 == 0]    → [0, 4, 16, 36]
[[e,e**2] for e in range(4) if e%2 != 0] → [[1,1], [3,9]]
```

# YOU TRY IT!

- What is the value returned by this expression?
  - Step1: what are **all values** in the sequence
  - Step2: which **subset of values** does the condition filter out?
  - Step3: **apply the function** to those values

```
[len(x) for x in ['xy', 'abcd', 7, '4.0'] if type(x) == str]
```

# DICTIONARIES

# HOW TO STORE STUDENT INFO

- so far, can store using separate lists for every info

```
names = ['Ana', 'John', 'Denise', 'Katy']
grade = ['B', 'A+', 'A', 'A']
course = [2.00, 6.0001, 20.002, 9.01]
```

- a **separate list** for each item
- each list must have the **same length**
- info stored across lists at **same index**, each index refers to info for a different person

# HOW TO UPDATE/RETRIEVE STUDENT INFO

```
def get_grade(student, name_list, grade_list, course_list):
    i = name_list.index(student)
    grade = grade_list[i]
    course = course_list[i]
    return (course, grade)
```

- **messy** if have a lot of different info to keep track of
- must maintain **many lists** and pass them as arguments
- must **always index** using integers
- must remember to change multiple lists

# A BETTER AND CLEANER WAY – A DICTIONARY

- nice to **index item of interest directly** (not always int)
- nice to use **one data structure**, no separate lists

**A list**

| | |
|---|---|
| 0 | Elem 1 |
| 1 | Elem 2 |
| 2 | Elem 3 |
| 3 | Elem 4 |
| … | … |

index element

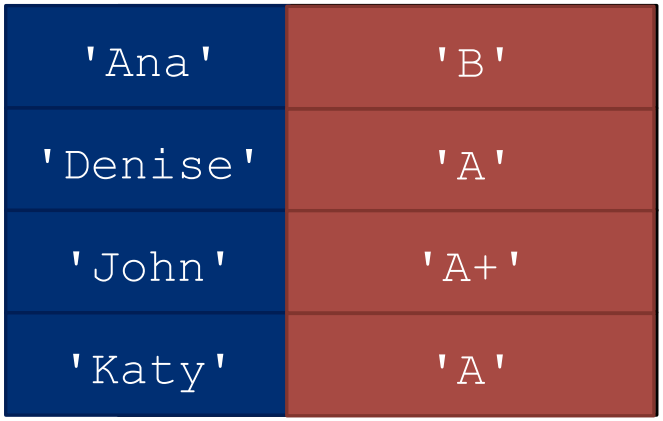
**A dictionary**

| | |
|---|---|
| Key 1 | Val 1 |
| Key 2 | Val 2 |
| Key 3 | Val 3 |
| Key 4 | Val 4 |
| … | … |

custom index by label element

# A PYTHON DICTIONARY

- store pairs of data
  - key
  - value




empty dictionary

```
my_dict = {}
```

```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
```

key1 val1 key2 val2 key3 val3 key4 val4

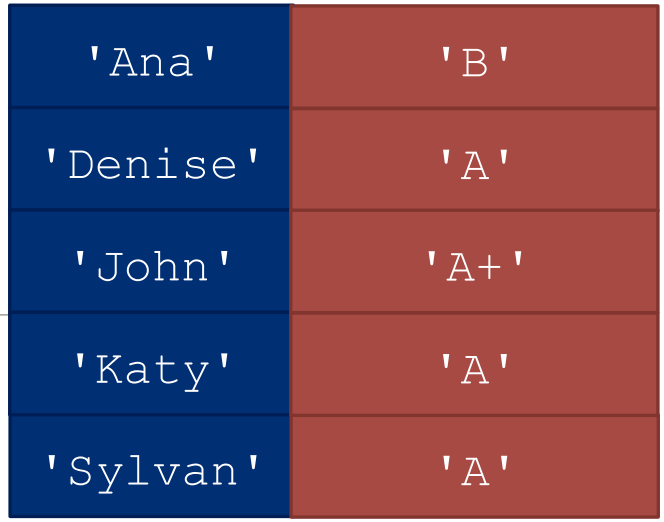
# DICTIONARY LOOKUP

- similar to indexing into a list
- **looks up** the **key**
- **returns** the **value** associated with the key
- if key isn't found, get an error

| | |
|---|---|
| 'Ana' | 'B' |
| 'Denise' | 'A' |
| 'John' | 'A+' |
| 'Katy' | 'A' |

```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
grades['John']      → evaluates to 'A+'
grades['Sylvan']    → gives a KeyError
```

# DICTIONARY OPERATIONS




```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
```

- **add** an entry

```
grades['Sylvan'] = 'A'
```

- **test** if key in dictionary

```
'John' in grades        → returns True
'Daniel' in grades      → returns False
```

- **delete** entry

```
del(grades['Ana'])
```

# DICTIONARY OPERATIONS

| | |
|---|---|
| 'Ana' | 'B' |
| 'Denise' | 'A' |
| 'John' | 'A+' |
| 'Katy' | 'A' |

```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
```

- get an **iterable that acts like a tuple of all keys** no guaranteed order

  `grades.keys()` → returns `['Denise','Katy','John','Ana']`

- get an **iterable that acts like a tuple of all values**

  `grades.values()` → returns `['A', 'A', 'A+', 'B']`

no guaranteed order

# DICTIONARY KEYS and VALUES

- values
  - any type (**immutable and mutable**)
  - can be **duplicates**
  - dictionary values can be lists, even other dictionaries!
- keys
  - must be **unique**
  - **immutable** type (`int`, `float`, `string`, `tuple`, `bool`)
    - actually need an object that is **hashable,** but think of as immutable as all immutable types are hashable
  - careful with `float` type as a key
- **no order** to keys or values!

```
d = {4:{1:0}, (1,3):"twelve", 'const':[3.14,2.7,8.44]}
```

# `list` vs `dict`

- **ordered** sequence of elements
- look up elements by an integer index
- indices have an **order**
- index is an **integer**

- **matches** "keys" to "values"
- look up one item by another item
- **no order** is guaranteed
- key can be any **immutable** type

# EXAMPLE: 3 FUNCTIONS TO ANALYZE SONG LYRICS

1) create a **frequency dictionary** mapping `str:int`

2) find **word that occurs the most** and how many times
   - use a list, in case there is more than one word
   - return a tuple `(list,int)` for (words_list, highest_freq)

3) find the **words that occur at least X times**
   - let user choose "at least X times", so allow as parameter
   - return a list of tuples, each tuple is a `(list, int)` containing the list of words ordered by their frequency
   - IDEA: From song dictionary, find most frequent word. Delete most common word. Repeat. It works because you are mutating the song dictionary.

# CREATING A DICTIONARY

```
def lyrics_to_frequencies(lyrics):
    myDict = {}
    for word in lyrics:
        if word in myDict:
            myDict[word] += 1
        else:
            myDict[word] = 1
    return myDict
```

can iterate over list

can iterate over keys in dictionary

update value associated with key

# USING THE DICTIONARY

```
def most_common_words(freqs):
    values = freqs.values()
    best = max(values)
    words = []
    for k in freqs:
        if freqs[k] == best:
            words.append(k)
    return (words, best)
```

this is an iterable, so can apply built-in function

can iterate over keys in dictionary

# LEVERAGING DICTIONARY PROPERTIES

```
def words_often(freqs, minTimes):
    result = []
    done = False
    while not done:
        temp = most_common_words(freqs)
        if temp[1] >= minTimes:
            result.append(temp)
            for w in temp[0]:
                del(freqs[w])
        else:
            done = True
    return result


print(words_often(beatles, 5))
```

can directly mutate dictionary; makes it easier to iterate

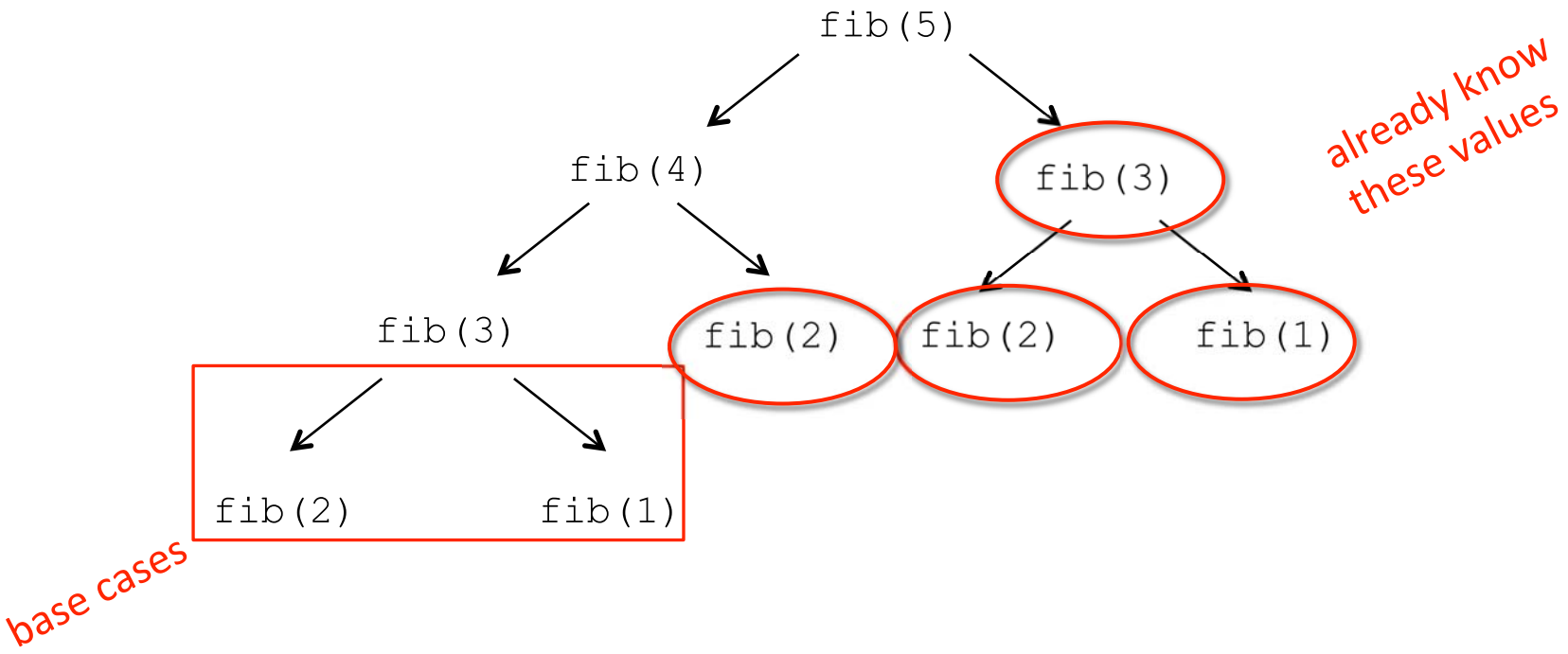
# FIBONACCI RECURSIVE CODE

```
def fib(n):
    if n == 1:
        return 1
    elif n == 2:
        return 2
    else:
        return fib(n-1) + fib(n-2)
```

- two base cases
- calls itself twice
- this code is inefficient

# INEFFICIENT FIBONACCI

`fib(n) = fib(n-1) + fib(n-2)`




- **recalculating** the same values many times!
- could keep **track** of already calculated values

# FIBONACCI WITH A DICTIONARY

```
def fib_efficient(n, d):
    if n in d:
        return d[n]
    else:
        ans = fib_efficient(n-1, d) + fib_efficient(n-2, d)
        d[n] = ans
        return ans

d = {1:1, 2:2}
print(fib_efficient(6, d))
```

Method sometimes called " memoization"

Initialize dictionary with base cases

- do a **lookup first** in case already calculated the value
- **modify dictionary** as progress through function calls

# EFFICIENCY GAINS

- Calling fib(34) results in 11,405,773 recursive calls to the procedure
- Calling fib_efficient(34) results in 65 recursive calls to the procedure
- Using dictionaries to capture intermediate results can be very efficient
- But note that this only works for procedures without side effects (i.e., the procedure will always produce the same result for a specific argument independent of any other computations between calls)

# 6.0001 Introduction to Computer Science and Programming in Python

Fall 2016